



UNIVERSITÀ DI PISA

DIPARTIMENTO DI INFORMATICA

RELAZIONE PROGETTO FARM

Maggio 2023

Autore:

Giuseppe Conte

Indice

1	Introduzione	2
2	Compilazione ed esecuzione su terminale	2
3	Uno sguardo generale	3
4	Il Main	3
5	Il Producer	3
6	Il Worker	3
7	Il Collector	3
8	Gestione della coda	4
9	Gestione dei segnali	4
10	Gestione del socket	4
11	Note	4

1 Introduzione

Farm è un programma composto da due processi, il primo denominato MasterWorker ed il secondo denominato Collector. MasterWorker, è un processo multi-threaded composto da un thread Master e da 'n' thread Worker.

Il programma prende come argomenti una lista (eventualmente vuota) di file binari contenenti numeri interi lunghi ed un certo numero di argomenti opzionali (le opzioni sono '-n', '-q', '-t', '-d'). Il processo Collector viene generato dal processo MasterWorker. I due processi comunicano attraverso una connessione socket AF_LOCAL (AF_UNIX).

Il processo MasterWorker legge gli argomenti passati alla funzione main uno alla volta, verificando che siano file regolari. Se viene passata l'opzione '-d' che prevede come argomento un nome di directory, viene navigata la directory passata come argomento e considerati tutti i file e le directory al suo interno.

Il nome del generico file di input viene inviato ad uno dei 'n' thread Worker. Il generico thread Worker si occupa di leggere dal disco il contenuto dell'intero file il cui nome ha ricevuto in input, di effettuare un calcolo sugli elementi letti e quindi di inviare il risultato ottenuto, unitamente al nome del file, al processo Collector.

Il processo Collector attende di ricevere tutti i risultati dai Worker ed al termine stampa i valori ottenuti sullo standard output, unitamente al nome del file, ordinando la stampa sulla base del risultato in ordine crescente.

2 Compilazione ed esecuzione su terminale

Per **compilare** il progetto su terminale linux è sufficiente eseguire il comando:
make

Per **generare** alcuni file in formato binario su terminale linux è sufficiente eseguire il comando:
make file

Per **eseguire** il programma su terminale linux è sufficiente eseguire il comando:
make test
(se nella cartella è presente anche l'eseguibile *generafile*)

oppure nel formato:
./farm -n <nthread> -q <qlen> -d <directory-name> -t <delay> "list of fileX.dat" -h

oppure:
make test2

Per pulire dalla cartella i file binari (generati col comando make file) e l'eseguibile è sufficiente digitare:
make clean

3 Uno sguardo generale

Il progetto è composto da 5 file scritti da me (main.c, masterworker.c, masterthread.c, collector.c, myutil.h), 4 file che sono quasi interamente le soluzioni delle esercitazioni delle lezioni (boundedqueue.c, boundedqueue.h, util.h, conn.h), generafile.sh (dove dal file test.sh ho copiato il codice relativo alla generazione dei file binari) e il makefile.

4 Il Main

Il file **main.c** contiene la funzione main del programma. Il main si occupa di leggere i parametri del programma che consistono in una lista di file binari e le opzioni '-n', '-q', '-t', '-d', '-h'. Ho aggiunto l'opzione '-h' che fa solo la stampa sullo standard output del formato di come scrivere i parametri di input, per poi terminare il programma.

Dopodichè il main maschera i segnali SIGINT, SIGQUIT, SIGTERM, SIGHUP, SIGUSR1, si ignora il segnale SIGPIPE e lancia la **fork()** per avviare il processo **Collector**.

Poi si avviano il thread handler per la gestione dei segnali e gli 'n' thread consumatori e **Worker**. Si avvia il **produttore**, si producono gli EOS e si aggiungono alla coda per terminare i thread consumatori, si termina l'handler e infine si libera la memoria.

5 Il Producer

La funzione **producer** è implementata nel file **masterthread.c**.

La funzione **producer** si occupa di aggiungere in coda i nomi dei file binari di cui effettuare il calcolo, ed eventualmente la fa la sleep di 't' millisecondi per rallentare il Master.

6 Il Worker

La funzione **Worker** è implementata nel file **poolworker.c**.

Diversi Worker sono eseguiti in parallelo su diversi thread.

La funzione **Worker** estrae dalla coda il nome del file binario, apre il file, ne legge i dati, effettua il calcolo richiesto ed invia il risultato e il nome del file al processo **Collector** tramite la connessione socket AF_LOCAL.

7 Il Collector

La funzione **collector** è implementata nel file **collector.c**.

Il collector viene eseguito su un processo differente rispetto alle altre funzioni. La funzione **collector** riceve i risultati dei calcoli e i nomi dei file tramite la connessione socket AF_LOCAL, salva i dati in una lista ordinata, li stampa sullo standard output e infine libera la memoria.

8 Gestione della coda

Per l'implementazione e la gestione della coda e della concorrenza ho usato il codice delle soluzioni delle esercitazioni. La coda è implementata come una lista finita di elementi di dimensione 'q'.

9 Gestione dei segnali

I segnali SIGINT, SIGQUIT, SIGTERM, SIGHUP, SIGUSR1 vengono gestiti da un thread handler avviato nel main. Alla ricezione di uno di questi segnali tranne SIGUSR1 viene settato a 1 il valore di una variabile globale atomica chiamata "*termina*", il cui valore viene sempre confrontato nel **Producer**, che ne fa terminare l'esecuzione.

Alla ricezione del segnale SIGUSR1 mediante il canale socket viene inviato al collector il messaggio "*sigusr1*".

10 Gestione del socket

Per l'implementazione del socket e delle funzioni *writn* e *reedn* ho usato il codice delle soluzioni delle esercitazioni.

Il processo **Collector** fa da Server, crea il socket, aspetta le connessioni e termina quando ha ricevuto un messaggio "*close*", che viene inviato dal **Main**.

I Client che si collegano al Server sono i **Worker** che inviano i risultati e i nomi dei file, il **main** e il sighandler, che inviano il messaggio "*close*" oppure "*sigusr1*".

Il Server ha un unico Thread che accetta tutte le connessioni e legge i messaggi.

11 Note

All'inizio del Worker ho inserito un **sleep(1)**, in modo da consentire al **collector** di partire prima, che crea il socket, altrimenti risultano problemi con la connessione.

Per verificare se i file binari sono regolari confronto se nel loro nome è incluso il segmento "**.dat**".

Non sono pienamente convinto di aver gestito bene la memoria e/o puntatori, eseguendo il programma con valgrind a volte vengono rilevati dei problemi, e spostando la *mySleep(t)* presente nel producer vengono rilevati molti problemi. Attualmente la *mySleep* non è inserita nella posizione giusta.

Se eseguo il programma col parametro "t" e invio un segnale di terminazione o di stampa parziale il programma non termina o stampa immediatamente.

Nel codice sono presenti diversi "#ifdef DEBUG printf(...); #endif" che ho usato per verificare/debuggare il codice man mano che lo scrivevo.

Per compilare il codice abilitando queste print bisogna eseguire: `make debug`.

Il codice è stato scritto e testato su una macchina multi-core Linux Ubuntu
21.04 hirsute